WORKLOOM · Backend

Work marketplace: quick tasks, agent tasks, gigs, contracts. Robinhood Chain (4663), USDG. Prepared 2026-10-16. (U) = unverified; confirm before the milestone that needs it.

Stack:

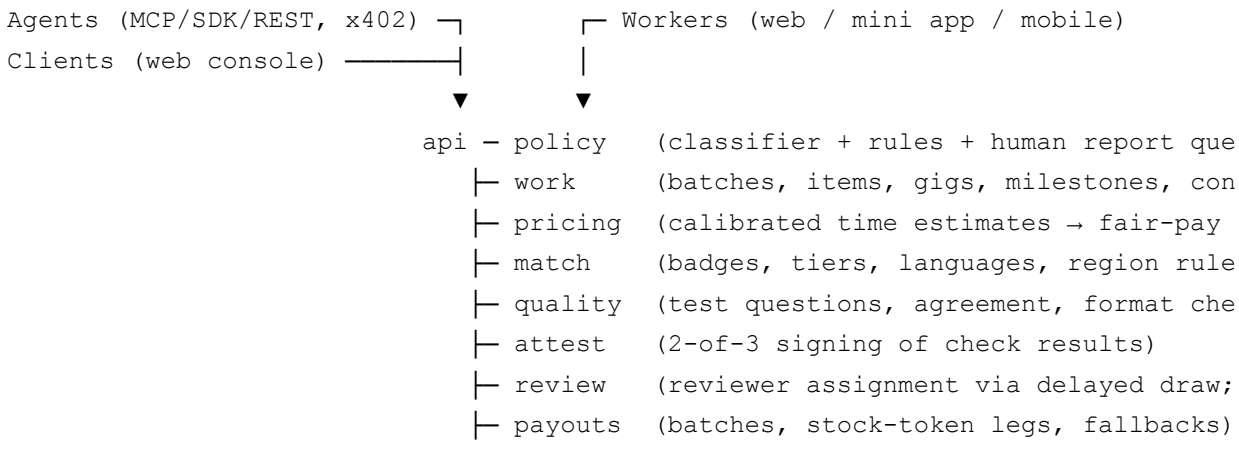
- **Backend:** TypeScript on Bun; Hono + tRPC; Drizzle on Postgres (+ Timescale); BullMQ on Redis; viem; Dwellir archive RPC.
- **Contracts:** Foundry; Solidity 0.8.x.
- **Accounts:** ZeroDev Kernel passkey accounts (EntryPoint v0.7; ZeroDev/Alchemy bundlers), with a second device or backup signer for recovery.
- **Clients:** web app (main), Telegram mini app (if Telegram's rules allow it, U), mobile shell for Local tasks.

Chain facts:

- **USDG:** no EIP-3009 or EIP-2612, so all funding uses **Permit2** with exact-amount signatures (no standing allowances).
- **Keepers:** no Chainlink Automation, so they're self-hosted, and every money path is also callable by anyone.
- **Gas:** the subsidy ended 2026-09-29, so payouts are batched and sponsored gas is capped.

---

1. Architecture



```
                                ├─ identity (WorkerPass, device attestation, tiers
                                └─ notify
On-chain: TaskEscrow · GigEscrow · PayRuns · PayoutRouter · WorkerPass · StakeVa
```

---

2. Work and policy

- **Task batch:**
  - type (Judge / Verify / Test / Label; Local later);
  - instructions;
  - items (hashes on-chain, content off-chain);
  - **k** workers per item;
  - test-question rate;
  - pay per assignment;
  - claim time limit;
  - review window (contract minimum **24h**);
  - region and language rules.
- **Policy** (classifier + rules + human report queue) runs on every post from every entry point. Refused:
  - illegal tasks;
  - anything about or locating individuals;
  - impersonation or calls on someone's behalf;
  - CAPTCHA or verification bypass;
  - fake reviews or engagement;
  - graphic or sexual content;
  - credential handling;
  - collecting personal data. The schema validator blocks contact, personal-data and seed-phrase answer fields;
  - pay-to-work;
  - moving off-platform;
  - installing apps or executables;
  - connecting wallets, signing transactions or entering payment details;
  - creating accounts on other services;
  - links failing a safe-browsing/sandbox check;
  - academic cheating;

    - regulated professional work without the worker's own (declared) licence;
    - Local tasks outside the allowed categories.
- **Test tasks:** web apps in a sandboxed browser, or store-distributed test builds, with requester-provided synthetic test accounts only.
- **Personal data in items:** scanned and redacted at post. The requester attests that items contain no personal data.
- **Fair-pay floor:**
    - **The estimate:** from Workloom-run calibration on independent Tier ≥ 1 workers. People who fail test questions and anyone clustered with the requester are excluded. The **65th-percentile** time is used.
    - **The floor:** below $6/hour, the batch can't list.
    - **During the run:** the realised rate is tracked. A breach prompts a `topUp`. With no top-up within 3 days, the unclaimed remainder is cancelled and refunded.

---

3. Quality engine

- **Test questions:**
    - mixed in at random; show-rate capped per item; rotated per worker; never shown to clustered workers together.
    - **Requester-written ones** are validated against agreement among high-accuracy workers. Contested ones are retired automatically.
- **Accuracy:** per task type. Below 80% over the last 50 test questions pauses that type, **with appeal** and retraining.
- **Pay is never cut by test or agreement results.** Test items are paid like any item. Results feed accuracy and confidence only, for any assignment that passed format checks.
- **Agreement:** weighted (Dawid–Skene style). Low-confidence items get one more assignment within budget. Answers on non-test items are correlated to catch copy rings.
- **Format checks:** schema and fields. Test screenshots must come from the tested app (U: metadata check). Local photos need signed capture (U), with faces and plates blurred.

---

4. TaskEscrow (quick and agent tasks)

```
interface ITaskEscrow {
    struct Batch {
        address requester; bytes32 specHash; bytes32 itemsRoot;
        uint32 assignments; uint96 payPer;
        uint96 feeReserve;        // 12% (minus verified $WLOOM discount), quote-
        uint96 reviewReserve;     // scales with batch size; requester-funded
        uint64 claimDeadline;
        uint64 expiresAt;         // ≥ claimDeadline + claim limit + reviewWindow
        uint32 reviewWindow;      // ≥ 24 h, enforced
        bytes32 drawSeed;         // fixed at create; used for reviewer draws
    }
    function create(Batch calldata b, bytes calldata quoteSig) external returns (
    function fund(uint256 id) external;                        // exact-amount Per
    function claim(uint256 id, bytes32 assignmentId) external;   // worker; time-
    function submit(uint256 id, bytes32 assignmentId, bytes32 answerHash, bool ch
        // stores submittedAt + checksPassed; locks payPer out of the refundable
    function approve(uint256 id, bytes32 assignmentId) external;            // re
    function approveDefault(uint256 id, bytes32 assignmentId) external;     // an
    function reject(uint256 id, bytes32 assignmentId, bytes32 reasonHash) externa
    function draw(uint256 id, bytes32 assignmentId) external;               // an
    function resolve(uint256 id, bytes32 assignmentId, bool workerWins, bytes cal
    function resolveDefault(uint256 id, bytes32 assignmentId) external;
        // after 72 h: checksPassed → worker credited; else one escalation (new d
    function cancelUnclaimed(uint256 id) external;            // requester: close
    function refundUnclaimed(uint256 id) external;            // anyone after exp
    function topUp(uint256 id, uint96 newPayPer, bytes calldata quoteSig) externa
}
```

Reviewers:

- **on-chain `ReviewerSet`, managed through Safe 2-of-3 + 48h;**
- **paid a flat fee per case, whatever they decide;**
- **the fee comes from the requester's review reserve; the reserve is returned only if the requester wins.**

Fees: taken only on credited assignments. Refunds include the fee share.

---

5. GigEscrow (gigs)

```
interface IGigEscrow {
    function post(bytes32 specHash, uint96[] calldata amounts, uint64[] calldata
                  uint32 reviewWindow, bytes calldata quoteSig) external returns
    function fundMilestone(uint256 gigId, uint8 i) external;   // amount + 8% fee
```

```
    function invite(uint256 gigId, address worker) external;   // client
    function accept(uint256 gigId) external;                   // worker: becomes
    function start(uint256 gigId, uint8 i) external;           // worker: marks m
    function progress(uint256 gigId, uint8 i, bytes32 partialHash) external; // w
    function submit(uint256 gigId, uint8 i, bytes32 deliveryHash) external;
    function approve(uint256 gigId, uint8 i) external;         // client; or anyo
    function dispute(uint256 gigId, uint8 i, bytes32 reasonHash) external;   // c
    function resolve(uint256 gigId, uint8 i, uint16 workerBps, bytes calldata ass
    function resolveDefault(uint256 gigId, uint8 i) external;  // one escalation,
    function extend(uint256 gigId, uint8 i, uint64 newDeadline, bytes calldata wo
    function cancelUnstarted(uint256 gigId, uint8 i) external; // client, only be
    function cancelOverdue(uint256 gigId, uint8 i) external;   // client, after d
                                                               //   no partialHas
}
```

---

6. PayRuns (contracts)

```
interface IPayRuns {
    function openTeam(address[] calldata admins, uint8 threshold) external return
    function fundTeam(uint256 teamId, uint256 amount) external;
    function offer(uint256 teamId, address worker, uint96 ratePerPeriod, uint32 p
    function acceptOffer(uint256 contractId) external;                 // worker;
    function reserveNext(uint256 contractId) external;                 // anyone:
    function run(uint256 contractId) external;                         // anyone
    function withdrawStreamed(uint256 contractId) external;            // worker:
    function proposeRate(uint256 contractId, uint96 newRate) external; // admins;
    function acceptRate(uint256 contractId) external;                  // worker
    function endContract(uint256 contractId) external;                 // admins:
    function withdrawTeam(uint256 teamId, uint256 amount) external;    // admins:
}
```

- **One `paidThrough` per contract**, used by both `run` and streaming, so nothing is paid twice.
- **Reserved before work:** a period that isn't reserved before it starts **pauses the contract**, and the worker is told before working.
- **Admins can never cut into reserved pay.**
- **Pay statements** are generated per run; they're not employment payslips.

---

7. PayoutRouter, WorkHistory, BonusVault

```
interface IPayoutRouter {
    function setElection(uint16 stockBps, address[] calldata tokens, uint16[] cal
        // default 0; allow-listed tokens + pool keys; eligible countries via sel
    function withdraw(uint256 amount) external;                              // worke
    function payoutBatch(address[] calldata workers) external;               // anyon
        // per worker try/catch; stock legs only if oracle fresh + market open +
        // exact-input swap, min-out ≥ oracle × (1 − 0.5%) with multiplier applie
        // batch notional ≤ pool-depth cap; failure → USDG
    function tip(bytes32 workRef, address token, uint256 amount) external;
        // only on credited work; ≤ 100% of that item's pay; sanctions-screened;
}
interface IWorkHistory {
    function addEntry(address worker, bytes32 entryCommit, uint8 kind) external;
    // entryCommit = hash(kind, client handle, skill, rating, amount, clusterFlag
}
interface IBonusVault {   // launch only, fixed budget, USDG only
    function claimInvite(address referee) external;   // pull; referee has ≥ 10 e
    function claimFirstTasks() external;              // pull; first 5 approved q
}
```

- **Bonuses never sit in the pay path:** escrows never call `BonusVault`, so pay can't break if it's empty.
- **Bonus caps:**
  - per WorkerPass;
  - 5 invites max per referrer;
  - referrer↔referee and client↔worker clusters excluded;
  - Tier 0 web-fallback accounts excluded.
- **Work history entries** between clustered client and worker are flagged. Share pages show the count of distinct clients.

---

8. Identity, anti-abuse, safety

- **WorkerPass:** device attestation proves a genuine app, **not a unique person**. So tiers, stakes and accuracy history carry the anti-sybil weight, and the web fallback gets Tier 0 only.
- **Tiers:** Tier 0 (no stake, capped volume); Tier 1–2 (USDG stakes of $10 / $50 + accuracy history). $WLOOM does **not** reduce stakes.
- **`StakeVault`:**
  - exit after 14 days;

    - at most **one** freeze per exit request, for a named fraud review, 30 days total, then auto-release;
    - slashes need 2-of-3 reviewers + one appeal, and are capped.
- **Sanctions:**
    - screened at WorkerPass mint and before `submit` attestation;
    - credited balances are never frozen;
    - flagged uncredited pay is held for ≤ 30 days during an appeal, or kept where the law requires, and never returned to the requester.
- **Fingerprints:** hashed; 90-day retention.
- **Local tasks:**
    - requesters get only geofence pass/fail + coarse time;
    - EXIF stripped;
    - repeat targeting of the same premises rate-limited.
- **Requester API:** shows pseudonymous worker IDs per batch.
- **Every task has "skip without penalty".** Harmful-content reports go to a human queue.

---

9. Payments in

- **USDG:** exact-amount Permit2 into the escrow.
- **x402 for agents:** `POST /v1/batches` with **payTo = TaskEscrow** and a Permit2 witness = batch hash, so create + fund are atomic.
- **Pay from Stock Tokens (clients):** one atomic user-op from the client's own account:
    1. exact-amount Permit2;
    2. exact-output swap to USDG (min-out bound by the oracle, multiplier applied once);
    3. reverts if the market is closed or the oracle is stale;
    4. `create` + `fund`.

    Escrow holds USDG only.

---

10. Data and retention

```
requesters (id, address, kind agent|human, rejection_rate, created_at)
workers    (id, address, pass_id, tier, stake, languages jsonb, region_code, accu
batches    (id, onchain_id, requester_id, type, spec jsonb, k, gold_rate, pay_per
items      (batch_id, item_id, content_ref, is_gold, gold_answer jsonb NULL, gold
assignments(id, batch_id, item_id, worker_ref, claimed_at, submitted_at, answer j
gigs       (id, onchain_id, client_id, spec jsonb, assignee, status) ; milestones
contracts  (id, team_id, worker, rate, period_s, notice_s, stream, paid_through,
reviews    (ref, reason_hash, reviewer, draw_block, decision, decided_at, tx)
payouts    (worker_id, ts, usdg, stock_legs jsonb, tx)
```

Retention:

- Task and gig content is deleted 60 days after close (export before then).
- Fingerprints are kept 90 days.
- Payment and commitment records are kept.

---

11. Red team

Attack / failure	Defence
Scam listings	Listed only when funded; quote-signed terms; policy on every post
Requester rejects good work, or picks a friendly reviewer	Delayed draw (blockhash after reject + seed from create); cluster exclusion; flat-fee reviewers; review reserve lost; public rejection rate
Client cancels after work starts	`cancelUnstarted` only before `start`; `cancelOverdue` with a grace period + partial-work split
Admin drains team balance before payday	Periods reserved before they start; `withdrawTeam` limited to the unreserved balance
Retroactive rate cuts	Rate changes need the worker's signature and apply next period
Bad test questions	Validated by agreement; retired when contested; pauses appealable; pay never cut
	USDG only; pull-based; ≥ 10 explicitly approved tasks +

Bonus farming	accuracy + 7 days; Tier ≥ 1; cluster exclusions; small caps; fixed budget
Fake work-history entries	Cluster flags; distinct-client count; entries only from escrow credits
Payout swap manipulation	Allow-list; fresh oracle + market hours; 0.5% bound; per-worker fallback; batch cap
Tips as a side channel	Only on credited work; capped at 100% of item pay; screened

---

12. Build order (Claude-Code-sized)

#	Milestone	Days
M1	`TaskEscrow` (claim/submit/approveDefault/delayed draw/resolveDefault/refunds/topUp) + `ReviewerSet` + fuzz	6
M2	`WorkerPass` + device attestation + tiers + `StakeVault`	4
M3	Policy service + pricing calibration + matching	5
M4	Quality engine (test questions, agreement, appeals) + results API	5
M5	Web app (feed, claim, submit, skip, pay history) + `PayoutRouter` (USDG) + pay handles	5
—	**W1–W2 launch**	—
M6	Agent API/MCP + x402 into escrow; Test/Label tasks	4
M7	`GigEscrow` (accept/start/progress/partial split/extend) + client dashboard	6
M8	`PayRuns` (reserve/run/stream/rate consent/notice) + pay statements	6
M9	Stock-token legs + holdings view + tips	4
M10	`WorkHistory` + share pages; `BonusVault`; Telegram mini app (if allowed)	5
M11	Local tasks	6

Gates:

- `approveDefault`, `resolveDefault`, `refundUnclaimed`, `run`, `reserveNext`, `payoutBatch` and `withdraw` all work with Workloom services stopped;
- fuzz properties:
  - no submitted or disputed assignment refunded;
  - no double pay on Pay runs;
  - admins can never touch reserved pay;
  - cancels never touch started milestones (except via `cancelOverdue`);
- the reviewer draw can't be predicted at reject time (test against a simulated sequencer);
- external review before batch, milestone or team caps go above $5,000.

---

13. Honest limits (engineering)

- **Workloom runs the attestors and the reviewer set.** Reviewers are paid flat fees.
- **The delayed draw depends on RHC block hashes, which the sequencer controls (U).** It's better than a draw in the same transaction, not perfect.
- **Device attestation can be beaten by farms.** Tiers, stakes and accuracy history limit the damage.
- **Capture signing, blurring and the Telegram path are (U)** until tested.